ODOO ADDONS

AWS S3 Cloud Storage Integration

Odoo 18.0 Enterprise & Community Module
Technical Reference, Architecture, & User Manual

AUTHOR / ORGANIZATION	Metamorphosis, Joyanto
MODULE VERSION	1.0.0
ODOO COMPATIBILITY	v18.0 (Community & Enterprise)
DOCUMENT DATE	July 14, 2026

Table of Contents

1. Introduction & Core Philosophy

As enterprise Odoo databases scale, local filestore size is often the primary driver of high storage costs, slow system backups, and complicated database restorations. The `aws_odoo_integration_cloudaddons` module addresses this overhead by integrating Odoo's attachment mechanism directly with AWS Simple Storage Service (S3).

The module operates under a **cloud-first paradigm**, enabling administrators to configure how files are stored on a per-model basis. By delegating storage to AWS S3, local filestore requirements can be slashed to virtually zero while maintaining complete accessibility of attachments within Odoo.

> **KEY CORE VALUE PROPOSITION**
>
> Reduces database and filestore disk space usage, secures company assets on durable AWS S3 storage, simplifies server replication/migration, and enables a live file browser inside Odoo.

Three Storage Modes (Configurable Per Model):

- **AWS S3 Only**: Binary files are uploaded to AWS S3, and the Odoo attachment's `store_fname` is replaced with an `s3://` URL pointer. The local filestore file is immediately deleted or bypassed, storing 0kb locally. When accessed, the binary stream is retrieved from S3 on-demand.
- **Dual Storage**: Files are stored simultaneously in Odoo's local filestore and replicated to AWS S3. This provides redundant backups and high-availability compliances.
- **Odoo Only**: Standard Odoo behaviour is maintained. No S3 operations occur for the configured model.

2. Technical Architecture & Inheritances

The module is designed with a lightweight and modular database schema. Rather than disrupting Odoo's default file storage, it overrides the standard `ir.attachment` model's core lifecycle hooks, providing a transparent abstraction layer.

2.1. Data Models Reference

The backend introduces five core database models to support bucket connections, model routing, log terminals, and explorer tracking:

Model Technical Name	Description	Key Fields
`aws.s3.bucket`	Represents individual AWS S3 bucket credentials and region targets.	`name`, `region_name`, `aws_access_key_id`, `aws_secret_access_key` (Status)
`aws.s3.attachment.rule`	Configures storage policies (S3 Only vs Dual) and root prefixes per Odoo Model.	`model_id` (Many2one `ir.model`), `storage_mode`, `root_folder`, `buc`
`aws.s3.log`	Maintains audit trail for uploads, downloads, deletions, and connections.	`name` (Operation), `attachment_name`, `res_model`, `status`, `message`
`aws.s3.dashboard`	Transient model calculating live stats, rendering HTML visual widgets.	`bucket_count`, `active_rule_count`, `success_sync_count`, `failed_sync_count`
`aws.s3.file.explorer`	Transient wizard displaying bucket items. Backed by item lines.	`bucket_id`, `folder_path`, `item_ids` (One2many items)

2.2. The ir.attachment Override Strategy

The core engine resides in the inheritance of `ir.attachment`. The following method overrides are responsible for S3 intercept actions:

A. Hooking Creation (`create`) & Writing (`write`)

When a user uploads a file, the `create` or `write` method intercept scans for an active rule for the target `res_model`. If an active rule is found, a temporary execution context is established:

```
ctx = dict(
    self.env.context,
    s3_storage_mode=rule.storage_mode,
    s3_bucket_id=rule.bucket_id.id,
    s3_key=s3_key,
    s3_res_model=res_model,
    s3_res_id=res_id or 0,
)
```

This context instructs the secondary data calculation hook (`_get_datas_related_values`) on how to handle the file content.

B. Intercepting Binary Processing (`_get_datas_related_values`)

Standard Odoo calls this method to determine where and how to write the attachment data. The custom module overrides this method to upload the data to AWS S3 via `boto3`:

```
client = bucket._get_s3_client(bucket)
client.put_object(
    Bucket=bucket.name,
    Key=s3_key,
    Body=data,
    ContentType=mimetype or 'application/octet-stream',
)
```

If `s3_only` is the storage mode, it writes an `s3://` prefix string in `store_fname` (e.g. `s3://1/Sales/PartnerName/SO001/doc.pdf`) and sets `db_datas = False`, preventing Odoo from writing a duplicate copy to the local filestore.

C. Custom Reading (`_file_read`)

When Odoo requests file access, `_file_read(fname)` is triggered. If `fname` starts with `s3://`, the method parses the bucket ID and the S3 key, retrieves the binary data from AWS S3, and returns the byte stream:

```
bucket = self.env['aws.s3.bucket'].sudo().browse(bucket_id)
client = bucket._get_s3_client(bucket)
response = client.get_object(Bucket=bucket.name, Key=s3_key)
return response['Body'].read()
```

If the file is not found locally under a dual storage model (due to a deleted filestore file), a self-healing fallback queries S3 using the attachment's saved key. This self-healing mechanism automatically repairs missing local filestore records.

D. Deletion Hook (`unlink`)

When an attachment is deleted from Odoo, the module intercepts the call, captures the S3 bucket details and key path, executes `super().unlink()`, and subsequently calls `client.delete_object` to clean up AWS S3. This ensures no orphaned objects are left on S3.

3. Dynamic Folder Routing (3-tier)

Rather than dumping all files into a single root folder, the module automatically generates a clean, readable 3-tier folder path in AWS S3 based on the active Odoo record. This provides native organization when viewing files directly in the AWS Console.

Special characters like slashes (`/`) in folder names, employee names, or customer names are replaced with underscores (`_`) to avoid corrupting prefix paths.

Odoo Model	Model Technical Name	S3 Routing Path Rule	Example S3 Key Path
CRM Lead	`crm.lead`	`[Root]/[Lead Name]/[Filename]`	`Sync/CRM/New_Pipeline_Deal/quote.pdf`
Sales Order	`sale.order`	`[Root]/[Customer Name]/[Order Ref]/[Filename]`	`Sync/Sales/Deco_Addict/SO0012/design.png`
Invoice / Move	`account.move`	`[Root]/[Customer Name]/[Invoice Ref]/[Filename]`	`Sync/Accounting/Deco_Addict/INV_2026_001/inv.pdf`
Purchase Order	`purchase.order`	`[Root]/[Vendor Name]/[PO Ref]/[Filename]`	`Sync/Purchase/Wood_Supplier/PO008/receipt.txt`
Employee	`hr.employee`	`[Root]/[Employee Name]/[Filename]`	`Sync/HR/John_Doe/contract.pdf`
Project Task	`project.task`	`[Root]/[Project Name]/[Task Name]/[Filename]`	`Sync/Projects/Website_Redesign/UI_Drafts/wireframe.pdf`
Helpdesk Ticket	`helpdesk.ticket`	`[Root]/[Ticket Name]/[Filename]`	`Sync/Helpdesk/Login_Error/error_log.txt`
Inventory Picking	`stock.picking`	`[Root]/[Picking Name]/[Filename]`	`Sync/Inventory/WH_OUT_0041/slip.pdf`
Default Fallback	*Any other model*	`[Root]/[Model Display Name]/[Record Name]/[Filename]`	`Sync/Others/res.partner/Azure_Interior/card.png`

4. AWS S3 & IAM Configuration

For the module to operate safely, the configured AWS Access Keys must have strict, scoped permissions. We strongly recommend configuring an IAM Policy that restricts the keys to a single bucket target, adhering to the principle of least privilege.

Recommended IAM Policy Document

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket",
        "s3:GetBucketLocation",
        "s3:HeadObject"
      ],
      "Resource": [
        "arn:aws:s3:::your-odoo-bucket-name",
        "arn:aws:s3:::your-odoo-bucket-name/*"
      ]
    }
  ]
}
```

5. Odoo Setup & Features Walkthrough

5.1. Bucket Setup & Connection Testing

Administrators must navigate to **AWS S3** → **Configuration** → **Buckets Setup**. Add the exact Bucket Name, AWS Region, Access Key, and Secret Access Key.

Clicking the **Test Connection** button triggers an API head check (`head_bucket`) against AWS. Upon success, Odoo updates the state to `Connected` and saves an audit entry in the log terminal.

5.2. Model Sync Rules

Under **AWS S3** → **Configuration** → **Sync Rules**, rules are defined for targeted models.

A **Bulk Migrate Attachments to S3** action is provided inside each sync rule form view. Clicking this button scans for historical attachments associated with the rule's model which are not yet on S3, uploads them sequentially, updates their database references, and deletes the local Odoo filestore copies if configured for S3 Only mode.

5.3. S3 File Explorer

The module features a built-in interactive file explorer under **AWS S3** → **File Explorer**. Users select a configured bucket to load directories and files in real-time.

- **Folders**: Displayed with a folder icon. Clicking "Open" updates the prefix and navigates into the sub-directory.
- **Files**: Shows the size (formatted in KB, MB, etc.) and date modified. Clicking "Download" streams the file locally. Clicking "Delete" permanently deletes the file from the AWS bucket.

5.4. Monitoring Dashboard & Activity Logs

The dashboard displays graphical KPI widgets showing active buckets, configured rules, successful uploads, and sync failures. The **Activity Logs Terminal** provides a detailed audit trail of all AWS API events, storing tracebacks for any failed connections or transfers.

6. Quality Assurance (QA) & Code Review

As part of a professional security and stability review, the module's codebase was scrutinized for potential bugs, logical conflicts, resource leaks, and performance concerns.

SECURITY CONCERN: PLAINTEXT CREDENTIALS

The `aws.s3.bucket` model stores `aws_access_key_id` and `aws_secret_access_key` as plaintext string fields in the database. While the view masks the secret with a password field, any SQL access or backup file reveals these credentials in plaintext.

CRITICAL LEAK: TEMPORARY ATTACHMENT GROWTH

In `aws.s3.file.explorer.item`'s `action_download_file` method, downloading a file from S3 pulls the binary stream into Odoo memory and creates a permanent Odoo attachment (`public=True`) to stream it to the user. These files are **never cleaned up**. If users download files frequently, this will cause massive database and storage bloat, defeating the purpose of S3 offloading.

6.1. Performance & Latency Audit

Because the S3 upload inside `_get_datas_related_values` occurs **synchronously** inside Odoo's main transaction thread, creating or writing attachments blocks the user session until AWS completes the upload request. If S3 experiences network latency, Odoo will lag significantly.

6.2. Thread-Safety & Transactions

If Odoo's SQL transaction fails or rolls back *after* an attachment is uploaded to S3, the Odoo record is rolled back, but the physical file remains uploaded to S3. This leads to orphaned files on AWS. In contrast, the `unlink` (deletion) logic correctly calls `super().unlink()` first, only deleting the S3 file once database deletion succeeds, preventing database-to-cloud inconsistencies.

7. Suggested Optimizations & Solutions

To address the issues identified in the QA review, the following code optimizations are recommended:

Optimization 1: Generate S3 Pre-signed URLs for Downloads

Instead of writing a temporary attachment in Odoo when downloading from the File Explorer, use AWS pre-signed URLs. This redirects the browser directly to S3 and expires in 60 minutes:

```
def action_download_file(self):
    self.ensure_one()
    if self.type != 'file':
        return
    explorer = self.explorer_id
    try:
        client = explorer.bucket_id._get_s3_client(explorer.bucket_id)
        presigned_url = client.generate_presigned_url(
            'get_object',
            Params={'Bucket': explorer.bucket_id.name, 'Key': self.key},
            ExpiresIn=3600
        )
        return {
            'type': 'ir.actions.act_url',
            'url': presigned_url,
            'target': 'new',
        }
    except Exception as e:
        raise UserError(_("Failed to generate download link: %s") % str(e))
```

Optimization 2: Batched Migration

To prevent worker timeouts when migrating large historical filestores, wrap the migration loop inside `aws_s3_attachment_rule.py` in chunks (e.g. 50 files per batch) or execute it via an asynchronous Odoo Cron job or Queue job.

Optimization 3: Encrypted System Parameters

AWS credentials should be stored in system parameters or encrypted using cryptography models in Odoo rather than direct plaintext database fields.